

제조데이터 분석과 최적화

제조 최적화(1)

김한진

gks359@cbnu.ac.kr

충북대학교 산업인공지능연구센터

CONTENTS

I

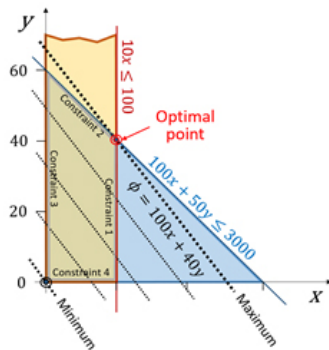
제조 최적화

II

제조 최적화와 실습

■ 공정 개선을 위한 이론

- **최적화 이론** : 주어진 제약 조건 안에서 발생할 수 있는 모든 대안들 중 가장 좋은 결과(최적해)를 찾아내는 수학적 방법론
 - ✓ 비용 최소화 혹은 이익 최대화를 이루는 수학적 모델 구축
- **스케줄링 이론** : 한정된 자원(Resource)을 여러 작업(Task)에 시간의 흐름에 따라 어떻게 할당할 것인가를 결정하여 특정 목표를 최적화하는 방법을 연구하는 분야
 - ✓ 공정 내 병목 현상 제거, 전체 작업 완료 시간 최소화, 납기 지연 방지
 - ✓ 최적화 이론의 하위 분야로 볼 수 있음(시간, 자원, 작업)



목적 함수:

$$\phi = 100x + 46y$$

초코빵은 개당 100원, 밀빵은 40원씩 이윤

제한 조건:

$$(1) 10x \leq 100$$

초코빵 만드는데 초콜릿 10g 필요 (재고 100g)

$$(2) 50y + 100x \leq 3000$$

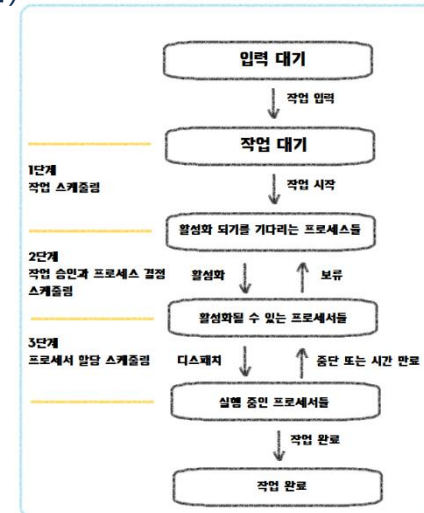
밀가루는 밀빵 만드는데 50g

초코빵에 100g 필요 (재고 3000g)

$$(3) x > 0$$

$$(4) y > 0$$

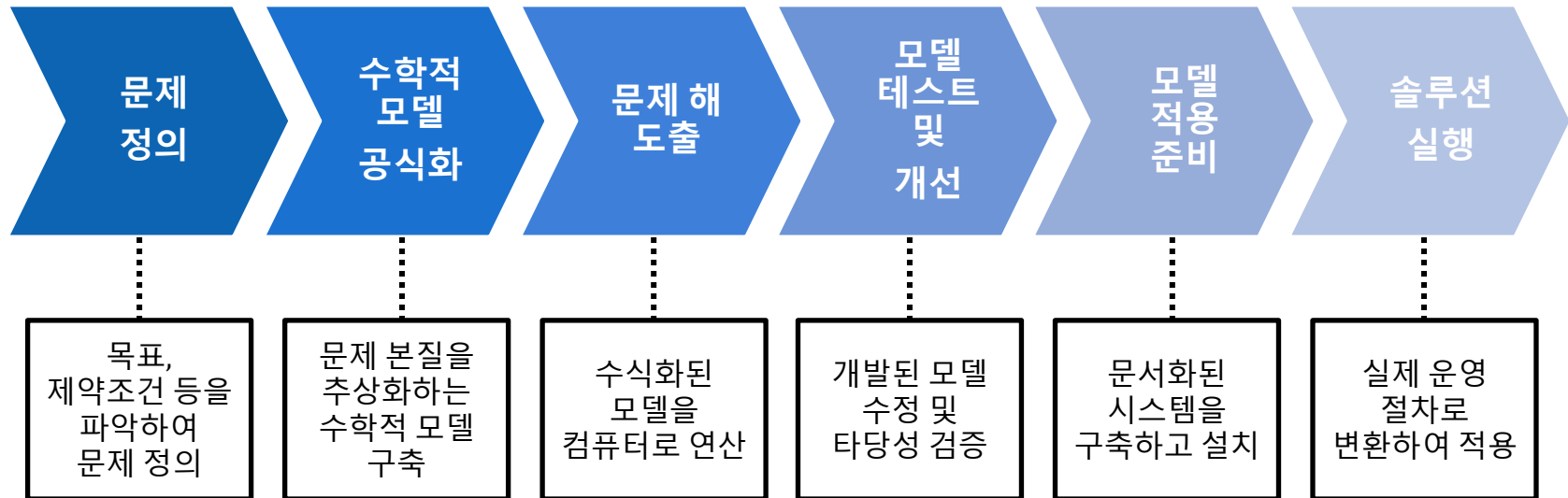
최적화 이론



스케줄링

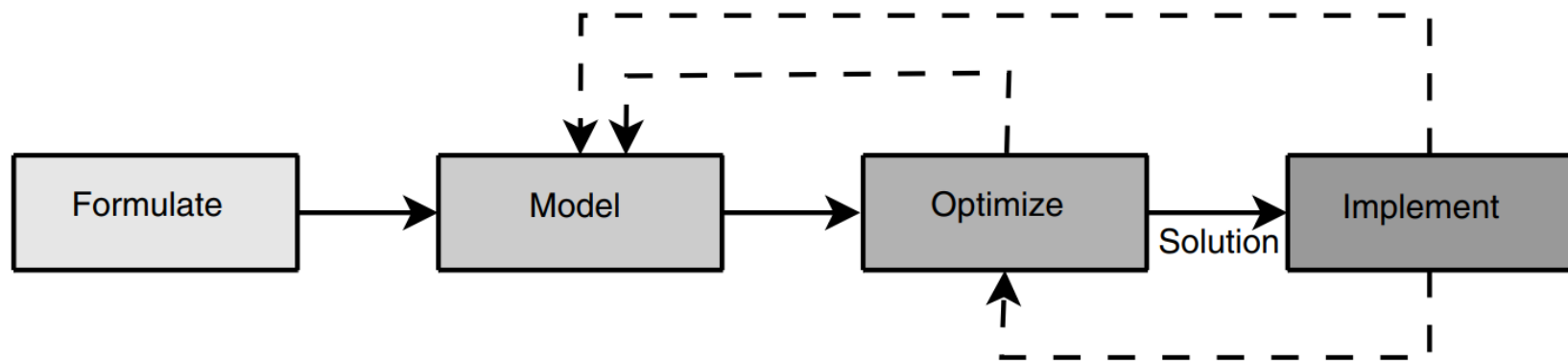
■ Operation Research의 최적화 프로세스

운영 연구(Operation Research, OR)는 경영 및 의사결정 개선을 위한
분석 방법 개발 및 적용을 다루는 응용 수학의 한 분야



■ 최적화 이론에서의 의사결정 프로세스

- Formulate the problem (문제 공식화)
의사결정 문제 식별
- Model the problem (문제 모델링)
문제에 대한 추상적인 수학적 모델 구축
- Optimize the problem (문제 최적화)
문제에 대한 최적해 생성
- Implement a solution (솔루션 구현)
솔루션의 구현



■ 수학적 모델

- 실제 문제를 이상적으로 표현한 것으로, 수학적 기호와 표현식을 통해 문제의 본질을 설명
주요 구성요소 :
 - ✓ **의사결정 변수** : 의사결정자가 정량화하여 궁극적으로 값을 결정해야 하는 대상
일반적으로 $x_1, x_2, \dots, x_n$ 과 같은 기호로 표현
예. 특정 제품의 생산량, 투자할 금액, 또는 각 활동의 실행 수준 등
 - ✓ **목적함수** : 의사결정 변수들을 통해 달성하고자 하는 목표나 성과 척도
일반적으로 변수 Z 로 표현
 - ✓ **제약조건** : 가용 자원의 한계나 정책적 규제 등, 의사결정 변수들이 가질 수 있는 값에 대한 물리적 또는 정책적 제한을 수학적 부등식($\leq, \geq$)이나 방정식(=)으로 나타낸 것
 - ✓ **매개변수** : 목적 함수나 제약 조건의 수식에 입력되는 상수들

$$\text{Maximize } Z = 3x_1 + 5x_2$$

Subject to :

$$x_1 \leq 4,$$

$$2x_2 \leq 12,$$

$$3x_1 + 2x_2 \leq 18$$

■ Linear Programming (선형 계획법)

제한된 자원을 가장 효율적으로 할당하여
원하는 목표(이익 최대화, 비용 최소화 등)를 달성하기 위한 수학적 최적화 기법

• 의사결정 변수 (Decision Variables):

- 문제 해결을 위해 우리가 최종적으로 값을 결정해야 하는 대상
- 주로 x_1, x_2 등으로 표기

• 목적 함수 (Objective Function):

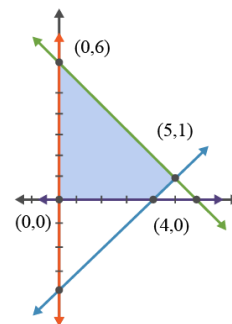
- 의사결정 변수들을 사용하여 최대화(Maximize) 또는 최소화(Minimize)하고자 하는 목표를 수학적 식으로 표현

• 제약 조건 (Constraints):

- 사용할 수 있는 자원의 한계나 반드시 지켜야 하는 규칙을 나타내는 부등식 또는 방정식
- 의사결정 변수는 일반적으로 0보다 크거나 같아야 한다는 비음 조건(Non-negativity constraint)을 가짐

$$\text{constraints: } \begin{cases} x + y \geq 6 \\ x - y \geq 4 \\ x \geq 0 \\ y \geq 0 \end{cases}$$

$$\text{Objective Function: } C = 2x + y$$



$$(0,0): C = 2(0) + (0) = 0 \text{ minimum}$$

$$(0,6): C = 2(0) + (6) = 6$$

$$(5,1): C = 2(5) + (1) = 11 \text{ maximum}$$

$$(4,0): C = 2(4) + (0) = 8$$

■ Linear Programming (선형 계획법)

목표: 이익 Z 를 최대화 (Maximize)

$$Z = 3x_1 + 4x_2$$

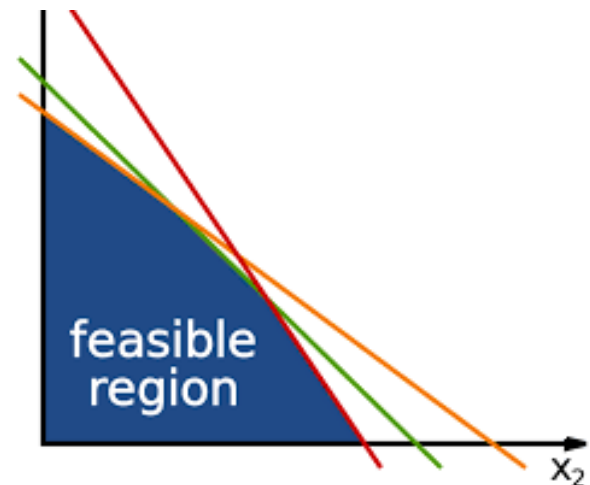
제약 조건: (Subject to)

$$x_1 + 2x_2 \leq 8$$

$$3x_1 + x_2 \leq 9$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$



위와 같이 모든 변수의 관계가 1차식(선형)으로 이루어져 있기 때문에 '선형' 계획법이라고 부름

■ Integer Programming (정수 계획법)

의사결정 변수의 일부 또는 전부가 반드시 '정수(Integer)'여야 한다는 조건이
추가된 수학적 최적화 기법

왜 필요? 반올림을 할 경우, 반올림한 값이 원래의 자원 한계를 초과하여 '실행 불가능한(Infeasible)' 해가 될 수 있음

• 순수 정수 계획법 (Pure Integer Programming):

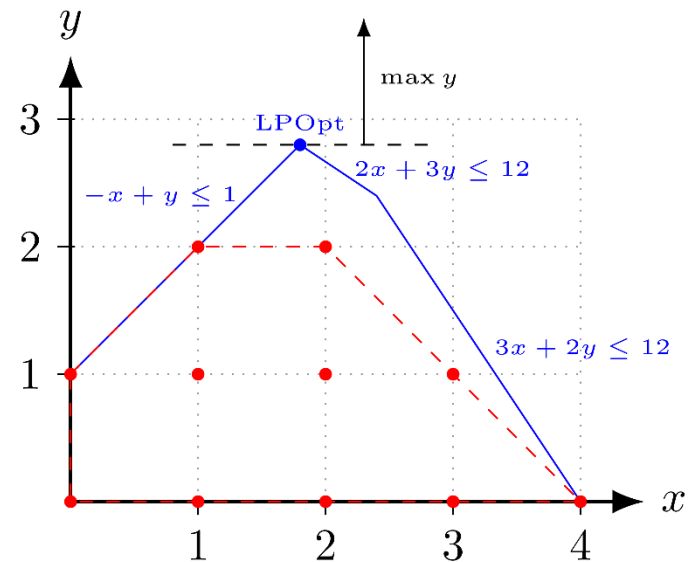
- 모든 의사결정 변수가 정수여야 하는 경우
- 예: 의자 x_1 개, 책상 x_2 개 생산 (둘 다 정수)

• 혼합 정수 계획법 (Mixed Integer Programming, MIP):

- 의사결정 변수 중 일부는 정수이고, 일부는 연속형 (소수점 가능) 변수인 경우
- 실무에서 가장 많이 쓰이는 형태 (예. 트럭 대수는 정수, 예산이나 연료량은 연속형 변수)

• 이진 정수 계획법 (Binary Integer Programming, 0-1 IP):

- 모든 변수가 0 또는 1의 값만 가질 수 있는 경우
- 'Yes or No', '선택할 것인가 말 것인가'를 결정하는 논리적 문제에 유용 (예. A 도시에 공장 지으면 1, 안 지으면 0)



■ 최적화 방법론 (Optimization Methods)

➤ Dynamic Programming

복잡한 문제를 여러 개의 간단한 하위 문제(Sub-problem)로 나누어 풀고, 그 결과를 저장해 두었다가 재사용하여 전체 문제를 해결하는 기법

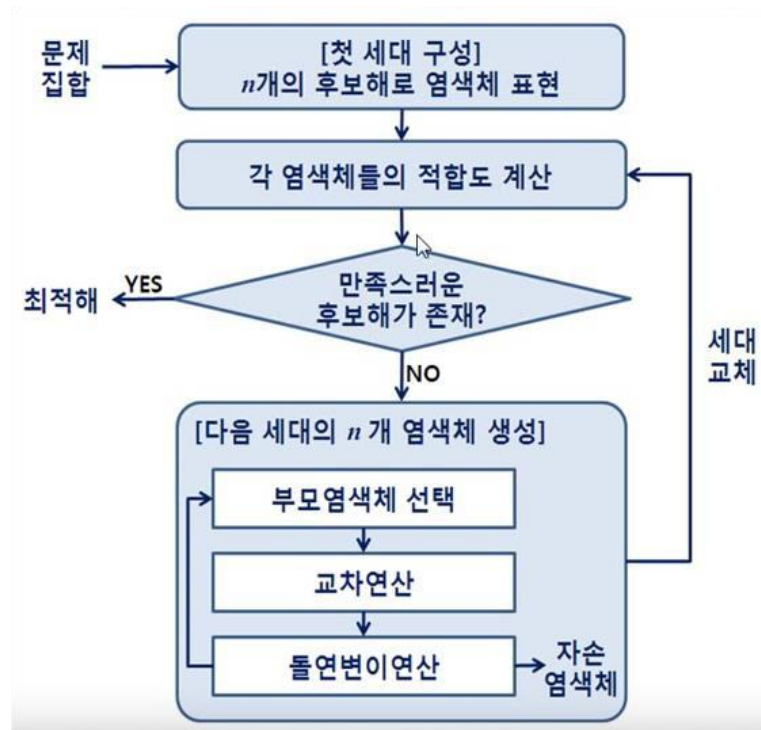
Knapsack Problem (Capacity = 7, 5 items)

	W	\$		0	1	2	3	4	5	6	7
			0	0	0	0	0	0	0	0	0
item 1:	3	50	1	0	0	0	50	50	50	50	50
item 2:	2	40	2	0	0	40	50	50	90	90	90
item 3:	4	70	3	0	0	40	50	70	90	110	120
item 4:	5	80	4	0	0	40	50	70	90	110	120
item 5:	1	10	5	0	10	40	50	70	90	110	120

■ 최적화 방법론 (Optimization Methods)

➤ 유전 알고리즘

찰스 다윈의 진화론에 등장하는 '자연선택'과 '적자생존'의 원리를 컴퓨터 프로그래밍에 적용한 최적화 방법



■ 가구 공장 생산 최적화

➤ 한정된 시간과 원자재 내에서 총 이익을 최대화하는 생산 조합은 무엇인가?



이익: 5만원

기계A: 2h 50%

기계B: 1h 25%

원자재: 3kg 75%

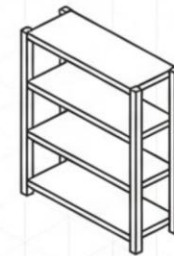


이익: 7만원

기계A: 3h 75%

기계B: 2h 50%

원자재: 2kg 50%



이익: 3만원

기계A: 1h 25%

기계B: 2h 50%

원자재: 4kg 100%

자원 제약 (Resource Constraints)

기계A 한계: 40시간

기계B 한계: 30시간

원자재 한계: 45kg

핵심 질문: 가장 이익이 큰 '책상'만 무조건 많이 생산하는 것이 최적의 해답일까?

■ 실습 알고리즘



완전 해법 (Exact Algorithms)

특징: 최적해 100% 보장
수학적 정식화 필수

LP (선형 계획법)

IP (정수 계획법)

DP (동적 계획법)



근사 및 AI 해법 (Heuristics & AI)

특징: 빠르고 유연한 탐색
비선형 및 복잡한 문제에 적합

Greedy (탐욕법)

GA (유전 알고리즘)

AI (베이지안 최적화)

■ 실습 환경 구축

➤ Anaconda 환경 생성

```
$ conda create -n mfg_data python=3.11 -y
```

```
$ conda activate mfg_data
```

➤ 필요 라이브러리 설치

```
$ pip install matplotlib pulp tabulate optuna
```

■ Linear Programming (1_lp_solver.py)

- 원리 : 이익 최대화를 목적함수로 설정하고, 자원한도를 제약 조건으로 정의하여 전역 최적해를 찾음
- 특징 : 변수가 실수(연속) 값을 가질 수 있으므로, 의자 12.5개와 같은 소수점 결과 도출 가능
→ 현실에서 12.5개의 가구를 만들 수 없으므로, 실제 제조 문제 적용 한계

① 결정 변수 (Decision Variables)

- x_1 : 의자(Chair) 생산량
- x_2 : 책상(Desk) 생산량
- x_3 : 선반(Shelf) 생산량
- (조건: $x_1, x_2, x_3 \geq 0$, 실수)

② 목적 함수 (Objective Function)

총 이익(Z)을 최대화합니다.

- Maximize $Z = 5x_1 + 7x_2 + 3x_3$ (단위: 만원)

③ 제약 조건 (Constraints)

사용 가능한 자원 한도 내에서 생산해야 합니다.

- 기계A (가공): $2x_1 + 3x_2 + 1x_3 \leq 40$ 시간
- 기계B (조립): $1x_1 + 2x_2 + 2x_3 \leq 30$ 시간
- 원자재: $3x_1 + 2x_2 + 4x_3 \leq 45$ kg

■ Linear Programming

➤ PuLP 파이썬 라이브러리

1단계 : 문제정의

- 가장 먼저 풀어야 할 문제의 빈 도화지를 만들
- 이때, 문제를 최대화할 것인지, 최소화할 것인지 방향을 정함

```
import pulp

# 문제의 이름과 목표(최대화)를 설정합니다.
prob = pulp.LpProblem("Production_Planning", pulp.LpMaximize)
```

- **pulp.LpMaximize**: 이익, 점수 등을 최대화할 때 사용
(이번 가구 공장의 목표)
- **pulp.LpMinimize**: 비용, 시간, 거리 등을 최소화할 때 사용

■ Linear Programming

➤ PuLP 파이썬 라이브러리

2단계 : 결정 변수 선언 (LpVariable)

- 우리가 구하고자 하는 미지수(x_1, x_2, x_3)를 만듦, 여기서는 '의자, 책상, 선반의 생산량'

```
# 의자 생산량 변수 생성 (0 이상, 연속/실수형)  
x_chair = pulp.LpVariable('x_chair', lowBound=0, cat='Continuous')
```

- lowBound=0: 생산량이 음수가 될 수는 없으므로, 최솟값을 0으로 고정
- cat='Continuous': 변수가 소수점을 포함한 실수 값을 가질 수 있도록 함 (LP의 핵심)
- 참고: 만약 cat='Integer'로 바꾸면, 이것이 바로 2번째 알고리즘이었던 정수 계획법(IP)

■ Linear Programming

➤ PuLP 파이썬 라이브러리

3단계: 목적 함수 설정 (Objective Function)

- 이 도화지(prob)에 목표 수식을 더함(+=). 우리의 목표는 총 이익을 구하는 것

```
# 목적 함수: 5*의자 + 7*책상 + 3*선반  
prob += (5 * x_chair + 7 * x_desk + 3 * x_shelf), "Total_Profit"
```

- 파이썬의 += 연산자를 사용하여 수식을 prob 객체에 직접 밀어 넣는 방식이 PuLP의 가장 큰 특징이자 장점
- 코드가 복잡할 경우, 리스트나 딕셔너리를 활용하여 pulp.lpSum() 함수로 수식을 한 번에 깔끔하게 더할 수 있음

■ Linear Programming

➤ PuLP 파이썬 라이브러리

4단계: 제약 조건 추가 (Constraints)

- 목적 함수와 똑같은 방식으로 += 연산자를 사용하되, 수식에 부등호(<=, >=, ==) 포함
- PuLP는 부등호가 있으면 이를 자동으로 '제약 조건'으로 인식

```
# 기계A 제약: 2*의자 + 3*책상 + 1*선반 <= 40
prob += (2 * x_chair + 3 * x_desk + 1 * x_shelf <= 40), "Machine_A_Constraint"

# 기계B 제약: 1*의자 + 2*책상 + 2*선반 <= 30
prob += (1 * x_chair + 2 * x_desk + 2 * x_shelf <= 30), "Machine_B_Constraint"
```

- 제약 조건 뒤에 "Machine_A_Constraint"처럼 이름을 적어두면, 나중에 디버깅을 하거나 수식을 출력할 때 알아보기 쉬움

■ Linear Programming

➤ PuLP 파이썬 라이브러리

5단계: 문제 풀이 (Solve)

- 변수, 목적 함수, 제약 조건이 모두 설정되었다면 솔버를 실행시켜 정답 도출

```
# CBC 솔버를 사용하여 풀이 실행  
prob.solve(pulp.PULP_CBC_CMD(msg=False))
```

- PuLP 자체는 문제를 정의하는 인터페이스 역할을 하며, 실제 계산은 내장된 기본 솔버인 CBC(Coin-or branch and cut)가 수행

■ Linear Programming

➤ PuLP 파이썬 라이브러리

6단계: 결과 확인 (Status & Values)

- 계산이 끝난 후, 문제가 정상적으로 풀렸는지 확인하고 정답 값 도출

```
# 풀이 상태 확인 (Optimal이 나오면 성공)
status = pulp.LpStatus[prob.status]

# 각 변수의 최적 값 확인
chair_qty = x_chair.varValue

# 목적 함수의 최종 결과값(최대 이익) 확인
max_profit = pulp.value(prob.objective)
```

- **pulp.LpStatus**: Optimal(최적해 찾음), Infeasible(제약 조건이 모순되어 해가 없음), Unbounded(이익이 무한대로 발산함) 등의 상태 반환
- **varValue**: 변수 객체에 담긴 최종 정답 숫자(생산량) 도출
- **pulp.value(prob.objective)**: 목적 함수에 정답을 대입했을 때 나오는 최종 계산 결과(총 이익) 도출

■ Dynamic Programming

- 1단계 (의자) : 전체 자원을 바탕으로 '의자를 몇 개 만들지(0개 ~ 최대치)' 하나씩 가정
- 2단계 (책상) : 의자를 만들고 남은 자원을 계산, 그 안에서 '책상을 몇 개 만들지' 하나씩 가정
- 3단계 (선반) : 의자와 책상 수량이 정해지면, 남은 자원은 명확

		책상 생산량 (Columns)														
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
의자 생산량 (Rows)	0	33	37	44	48	55	59	66	70	77	81	85	89	93	94	
	1	35	42	46	53	57	64	68	75	79	83	87	91	95		
	2	37	44	48	55	59	66	70	77	81	88	92	96	94		
	3	42	46	53	57	64	68	75	79	86	90	94	95			
	4	44	48	55	59	66	70	77	81	88	92	96				
	5	46	53	57	64	68	75	79	86	90	97	95				
	6	48	55	59	66	70	77	81	88	92	96					
	7	53	57	64	68	75	79	86	90	97						
	8	55	59	66	70	77	81	88	92	96						
	9	57	64	68	75	79	86	90	97							
	10	59	66	70	77	81	88	92								
	11	64	68	75	79	86	90	97								
	12	66	70	77	81	88										
	13	68	75	79	86											
	14	70	77													

■ Dynamic Programming

➤ 1단계 의자 생산량 반복

```
max_x0 = min(limit_a // usage_a[0], limit_b // usage_b[0], limit_m // usage_m[0])  
for x0 in range(max_x0 + 1):
```

- 제약 조건만 고려하여 의자를 최대 몇 개(max_x0) 만들 수 있는지 계산한 뒤,
0개부터 최대치까지 하나씩 시도

■ Dynamic Programming

➤ 2단계 남은 자원 계산 및 책상 생산량 반복

```
rem_a = limit_a - usage_a[0] * x0 # 의자를 만들고 남은 기계A 시간  
# ... (기계B, 원자재도 동일하게 계산)  
  
max_x1 = min(rem_a // usage_a[1], ...) # 남은 자원 기준 책상 최대치  
for x1 in range(max_x1 + 1):
```

- 의자를 x0개 만들었다고 가정하고, 남은 자원(rem_a, rem_b, rem_m)을 갱신
- 이 남은 자원을 기준으로 책상을 몇 개 만들 수 있는지(max_x1) 다시 계산하여 반복
- 불가능한 경우의 수는 Pruning(가지치기)

■ Dynamic Programming

➤ 3단계 선반 자동 결정 (반복문 생략, DP 핵심)

```
rem_a2 = rem_a - usage_a[1] * x1 # 책상까지 만들고 남은 최종 자원  
  
# 선반(x2)은 루프를 돌지 않고, 남은 자원으로 만들 수 있는 최대치를 바로 계산!  
x2 = min(rem_a2 // usage_a[2], ...)
```

- 의자와 책상의 수가 정해졌으니, 남은 자원(rem_a2 등)은 확정
- 선반은 남은 자원을 모두 털어서 만들 수 있는 최대 수량 계산 (3중 for loop 제외)

■ Greedy 알고리즘

- 전체적인 큰 그림을 보거나 미래를 예측하지 않고, "현재 상황에서 가장 최적이라고 생각되는 선택(지역적 최적해)"을 반복, 최종 해답에 도달하는 직관적이고 빠른 최적화 기법
- 탐욕적 선택 기준
 - 자원 소모 부담률 대비 이익(효율)

각 자원의 희소성(전체 한도)을 고려하여 정규화된 소모 비율 계산

$$Efficiency = \frac{Profit}{\sum(\frac{Usage}{Limit})}$$

■ Greedy 알고리즘

➤ 효율 계산

책상의 효율 계산 예시:

- 기계A 소모 비율: $3 \div 40 = 0.075$
- 기계B 소모 비율: $2 \div 30 = 0.067$
- 원자재 소모 비율: $2 \div 45 = 0.044$
- 총 자원 부담률 = 0.186
- 책상 효율 = $7 \div 0.186 \approx 37.6$ (1위)

효율 순위 : 책상 > 의자 > 선반

■ Greedy 알고리즘

➤ 코드 구현

1) 제품별 효율 계산 및 정렬 (`efficiency.sort`)

- 모든 제품의 효율성을 계산한 뒤, 가성비가 가장 높은 순서대로 내림차순 정렬 수행

2) 최우선 제품 최대 생산 (`max_possible`)

- 1위인 '책상'을 현재 남은 자원(기계A 40, 기계B 30, 원자재 45)이 허락하는 한계까지 생산
- 결과: 책상 13개 생산 (자원은 여유로우나, 기계A가 39 소모 제한)

3_ 자원 차감 및 차순위 반복 (`remaining[key] -= ...`)

- 남은 자원(기계A 1시간)을 가지고 2위인 '의자' 생산 시도, but 기계A 부족으로 0개 생산
- 마지막 남은 자원으로 3위인 '선반'을 1개 생산하고 탐색 종료

■ Genetic Algorithm, 유전 알고리즘

- 찰스 다윈의 '자연선택(Natural Selection)'과 '유전(Genetics)' 법칙을 모방하여, 수많은 후보 해답들이 세대를 거듭하며 스스로 최적해를 향해 진화하도록 만드는 기법
 - 특징 : 수식으로 완벽히 풀기 어려운 비선형 문제에 유연하게 적용 가능
 - 단점 : 전역 최적해를 100% 보장하지 않으며, 난수에 의존하므로 실행할 때마다 결과나 도달 시간이 다를 수 있음

■ Genetic Algorithm, 유전 알고리즘

➤ 개체(Individual)와 적합도(Fitness)

- 유전자 : [의자 생산량, 책상 생산량, 선반 생산량] 리스트 : 하나의 유전자(염색체)
- 적합도(생존력) : 생명체가 얼마나 우수한지 평가
 - $\text{적합도} = \text{총 이익} - (\text{초과자원량} \times 50)$
 - 제약 조건을 위반하면, 강력한 페널티(-50)를 부여, 자연 도태 유도

➤ 엘리트 보존 (Elitism)

- 방어적 생존 전략으로, 전체 1,2등 개체는 다음 세대로 그대로 활용
 - 선택, 교차, 변이의 과정 면제

■ Genetic Algorithm, 유전 알고리즘

➤ 선택 (Tournament Selection)

- 우수 유전자를 다음 세대에 남기기 위한 부모를 고르는 과정
 - 무작위로 3개 개체 뽑아 토너먼트에 올리고, 그 중 적합도가 가장 높은 부모 선택

➤ 교차 (Uniform Crossover)

- 두 부모의 유전자를 섞어 자식을 만듦 (80% 확률)
- 의자, 책상, 선반 각각에 대해 50% 확률로 엄마의 유전자를,
50% 확률로 아빠의 유전자를 물려 받음

➤ 변이 (Mutation)

- 자식 유전자 일부를 무작위로 변경 (15% 확률)
- 생산량을 기존 값에서 $\pm 30\%$ 범위 내 튀는 값으로 변경
 - 지역 최적해 빠지는 것 방지
 - 새로운 가능성 탐색

■ AI/Optuna 최적화 알고리즘

- 문제의 복잡한 수학적 수식이나 제약 조건을 컴퓨터에게 일일이 가르쳐주지 않고, 철저히 '블랙박스(Black-box)' 취급하여 최적해를 찾아내는 최신 인공지능 기반 탐색 기법
 - 특징 : 함수(공장 환경) 내부가 어떻게 생겼는지 몰라도, "입력값(생산량)"을 넣었을 때 나오는 "출력값(점수)"의 패턴만을 학습하여 정답을 유추
 - 활용 라이브러리 : 하이퍼파라미터 튜닝을 위해 Optuna 프레임워크 사용

■ AI/Optuna 최적화 알고리즘

- Optuna는 머신러닝 및 딥러닝 모델 하이퍼파라미터 최적화를 자동화하고 효율적으로 수행할 수 있게 도와주는 파이썬 기반 오픈소스 프레임워크

단순 무작위 탐색(Random Search)이 아니라, 과거의 시도(Trial) 결과를 바탕으로 다음번에는 더 성능이 좋을 것 같은 하이퍼파라미터 값을 추론하여 탐색

- **Trial (시도):** 목적 함수를 한 번 실행하는 과정
(예: "의자 3개, 책상 5개, 선반 2개를 만들어보자!")
- **Objective Function (목적 함수):** Trial에서 제안한 값을 바탕으로 점수(여기서는 이익)를 계산하여 Optuna에게 알려주는 함수
- **Study (연구):** 목적 함수를 여러 번(Trial) 반복하면서 최적의 해를 찾아가는 전체 프로젝트(세션)를 의미

■ AI/Optuna 최적화 알고리즘

➤ 목적 함수(_objective) 정의와 탐색 공간 설정

```
def _objective(self, trial):  
    # — 1. AI에게 각 제품의 생산량을 제안받음 —  
    quantities = []  
    for i, name in enumerate(self.env.products):  
        max_q = self.env.get_max_quantity(i)  
        # Optuna야, 0부터 최대 생산 가능량 사이에서 정수 하나를 추천해줘!  
        q = trial.suggest_int(f'x_{name}', 0, max_q)  
        quantities.append(q)
```

- trial.suggest_int(name, low, high): Optuna의 강력한 기능
- 변수의 이름을 지정하고 탐색할 범위를 주면, Optuna가 내부 알고리즘에 따라 값을 제안

■ AI/Optuna 최적화 알고리즘

➤ 제약 조건 처리 (페널티 부여)

```
# —— 제약 위반 시 소프트 페널티 부여 ——  
penalty = 0  
for usage in result['resource_usage'].values():  
    if usage['over'] > 0:  
        penalty += usage['over'] * 100 # 초과량에 비례한 강한 페널티
```

- Soft Penalty: Optuna가 제약 조건을 어기는 생산량을 제안했다면, 감점(페널티)을 크게 주어 "이 방향은 좋지 않다"고 학습시키는 방식 사용

➤ 제약 조건 처리 (페널티 부여)

```
# Optuna는 minimize가 기본 → 이익을 음수로 반환 (최소화 = 이익 최대화)  
return -(result['profit'] - penalty)
```

- 우리가 원하는 목표(이익 최대화)에 마이너스(-)를 붙여 반환

■ AI/Optuna 최적화 알고리즘

➤ Study 생성 및 최적화 실행 (solve 메서드)

```
# Study 생성
study = optuna.create_study(
    direction='minimize', # 최소화 방향으로 탐색 (-이익을 최소화)
    sampler=optuna.samplers.TPESampler(seed=42)
)

# 최적화 실행 (500번 반복)
study.optimize(self._objective, n_trials=self.n_trials)
```

- **optuna.create_study(...): Study 객체 생성**
 - **direction='minimize':** 반환되는 값을 가장 작게 만드는 것이 목표
 - **sampler=TPESampler(...):** TPE(Tree-structured Parzen Estimator) 알고리즘 사용
무작위로 값을 던지는 것이 아니라, 이전 Trial들의 성공/실패 데이터를 바탕으로
다음 번엔 더 높은 확률로 정답이 있을 만한 곳 선택
- **study.optimize(함수, 반복횟수): Optuna 엔진 가동**
 - **_objective** 함수를 500번 반복하면서 스스로 학습

■ AI/Optuna 최적화 알고리즘

➤ 최적 결과 추출

```
# 최적 결과 추출  
best_params = study.best_params  
quantities = [best_params[f'x_{name}']] for name in self.env.products]
```

- 탐색이 모두 끝나면 `study.best_params` 딕셔너리에 가장 좋은 점수를 냈던 변수값 저장

■ Optuna 정리

➤ 제안과 평가

- AI가 임의의 숫자를 제시 [trial.suggest_int] (예. 의자 10개, 책상 5개, 선반 3개)
- [env.evaluate]에 해당 숫자를 넣어 실제 이익과 자원 사용량 계산

➤ 벌점 부여

- 자원 한도를 초과하면, 초과한 수치에 100이라는 가중치를 곱해 벌점 부여
- 결과에 대한 피드백을 부여하여, AI 스스로 자원 한계선을 학습하도록 유도

➤ 베이지안 확률 학습

- 과거 채점 기록을 분석하여 다음 숫자 예측 (베이지안 최적화)

■ AI/Optuna 최적화 알고리즘

➤ 베이지안 최적화?

- 무작정 값을 넣어보는 대신, 과거의 데이터를 바탕으로 '미지의 함수 모습을 추론'하고, '다음에 어디를 찌러볼지 전략적으로 결정'

→ 1. 대리모델

현재까지 모은 데이터를 바탕으로 실제 함수와 비슷하게 생긴 가짜 함수(대리 모델) 만듦

- **평균** : 지금까지의 데이터로 볼 때, 이 지점의 값은 대략 이럴 것이다 라는 예측값
- **불확실성** : 하지만 이 지점은 데이터를 본 적이 없어 예측이 틀릴 수 있다
(관측된 데이터 근처는 불확실성이 낮고, 데이터가 없는 곳은 불확실성 높음)

→ 2. 획득함수

대리 모델이 예측한 평균과 불확실성을 조합, 다음 어느 지점 탐색하는 것이 가장 이득일 것인지 점수를 매기는 함수

- **활용** : 이미 값이 높게 나왔던 곳 근처를 파보는 전략
- **탐색** : 아직 한 번도 가보지 않아서 모르는 곳을 파보는 전략

감사합니다

Q&A

